

treenity — Code & Process Walkthrough

A guided explanation of every file in the repository, how the pieces fit together, and how to build/run/test the project

September 15, 2026

Contents

1	1. What this document is	2
2	2. gitinette — the git workflow guideline checker	2
2.1	2.1 What it checks	2
2.2	2.2 Two modes	3
2.3	2.3 How this project’s own history was built	3
3	3. Build system — Makefile	4
4	4. Shared utilities — src/common/	4
4.1	4.1 protocol.hpp — constants and wire-format helpers	4
4.2	4.2 ipc_util.hpp — FIFO plumbing	5
5	5. Data structures — src/common/	5
5.1	5.1 hashmap.hpp — the client-metadata hashmap	5
5.2	5.2 trie.hpp — the prefix-matching structure	5
6	6. Server — src/server/	6
6.1	6.1 topic.hpp — one worker thread per topic	6
6.2	6.2 server.cpp — control loop and request handlers	7
7	7. Client — src/client/client.cpp	11
7.1	7.1 create / list / info	11
7.2	7.2 produce [--raw]	11
7.3	7.3 subscribe <topic> <name> [--prefix P] [--offset N] [--raw]	11
8	8. A full request walked end to end	12
9	9. Testing strategy	13
9.1	9.1 Unit tests (tests/test_trie.cpp, tests/test_hashmap.cpp)	13
9.2	9.2 Integration test (tests/integration_test.sh)	13
10	10. Exit codes reference	14
11	11. Known limitations and deliberate trade-offs	14

1. What this document is

This document is a companion to README.md: the README is written for someone who wants to *use* the project (build it, run it, understand the high-level design decisions and why they were made). This document is written for someone who wants to *read* the project — it walks through every source file, explains what it does and why it is structured the way it is, and traces a full request through the system end to end. It also documents the two supporting processes around the code itself: the guideline-checking tool (gitinette) and the test/build pipeline (Makefile).

Repository layout:

```
treenity/
├── gitinette                # git workflow guideline checker (provided)
├── Makefile                 # all / clean / fclean / re / test
├── README.md               # user-facing documentation
├── .gitignore
├── src/
│   ├── common/
│   │   ├── hashmap.hpp     # hand-rolled client-metadata hashmap
│   │   ├── trie.hpp        # prefix trie + PrefixIndex
│   │   ├── protocol.hpp    # wire-format constants & helpers
│   │   └── ipc_util.hpp     # FIFO create/read/write helpers
│   ├── server/
│   │   ├── topic.hpp       # per-topic worker thread + consumer dispatch
│   │   └── server.cpp       # control loop, request handlers, main()
│   └── client/
│       └── client.cpp       # CLI: create/list/produce/subscribe/info
├── tests/
│   ├── test_trie.cpp        # GoogleTest: prefix trie / PrefixIndex
│   ├── test_hashmap.cpp     # GoogleTest: client-metadata hashmap
│   ├── integration_test.sh  # black-box test against real binaries
│   └── raw_helpers.py       # binary-framing helper for the --raw test
```

2. gitinette — the git workflow guideline checker

gitinette is a standalone Python script (not linked to the C++ build at all) that audits a git repository against the project's Git Workflow Guidelines: commit message format, branch naming, and merge strategy.

2.1 What it checks

- **Commit messages** (check_commit_messages): every *non-merge* commit subject must match

```
COMMIT_RE = re.compile(
    r"^(feat|fix|chore|refactor|test|docs|style|perf|build|ci|revert)"
    r"(\([[\w\-.\/]+\])?!?: .+)"
)
```

i.e. Conventional Commits: a known type, an optional (scope), an optional ! for breaking changes, then : and a summary. Merge commits (identified by having more than one parent) are explicitly skipped — they're allowed to have a free-form message like merge: integrate X.

- **Branch names** (`check_branch_names`): every local branch other than the base branch / main / master must match `<type>/<name>` using the same type vocabulary as commits, e.g. `feat/producer-client`.
- **Merge strategy** (`check_merge_strategy`), reported as *warnings* rather than hard violations:
 - if the base branch's history is fully linear (no merge commits) and has more than 5 commits, it warns that this looks like fast-forward or squash merging rather than `git merge --no-ff`;
 - it also always reports how many commits have exactly one parent (i.e. every non-merge commit, whether it originated on a feature branch that was later merged in, or was committed directly to the base branch) so a human can sanity-check that any *truly* direct commits on the base branch were small, safe changes.

This second warning is a known source of (harmless) noise: because `git log <base>` walks the *entire* ancestry, a commit made on a feature branch and properly merged with `--no-ff` still shows up with exactly one parent once it's reachable from the base branch — gitinette cannot distinguish “committed directly to main” from “committed on a branch that was merged into main” using this simple parent-count heuristic. That is why the subject frames this specific check as a warning requiring manual review, not a hard violation, and it is reflected in gitinette's own exit code: only entries under *Violations*: affect the exit code (return 1 if violations else 0); entries under *Warnings*: do not.

2.2 2.2 Two modes

- **Self-check mode** (no flags beyond `--repo/--base-branch`): prints violations and warnings, used by a student during development.
- **--evaluation mode**: in addition to the above, prints a random sample (`--sample N`, default 3) of non-merge commits with their full message and a `--stat` summary of changed files, via `git show --stat --format=%B <hash>`, so a human evaluator can judge whether the commit message actually describes the diff.

2.3 2.3 How this project's own history was built

gitinette `--repo . --base-branch main` passes with **zero violations** against this repository's own history. The history was deliberately built as one feature branch per logical unit of work, each merged into main with `git merge --no-ff` (never fast-forwarded, never squashed), for example:

```
* merge: integrate README
|\
| * docs: add README with architecture, IPC choice, ... sections
|/
* merge: integrate test suite and build system
|\
| * test: add gtest unit tests, integration script and Makefile
|/
* merge: integrate CLI client
|\
| * feat(client): implement create/list/produce/subscribe/info CLI
|/
* merge: integrate message-queue server
|\
| * feat(server): implement per-topic worker and control loop
|/
```

```

* merge: integrate common protocol/data-structure module
|\
| * feat(common): add hashmap, trie and wire protocol helpers
|/
* chore: add gitinette guideline checker
* chore: add gitignore for build artifacts

```

Every branch name (feat/protocol, feat/server, feat/client, test/unit-and-integration, docs/readme) matches <type>/<name>; every non-merge commit message matches Conventional Commits; every feature branch was integrated with an explicit merge commit.

3 3. Build system — Makefile

The Makefile exposes exactly the targets a evaluator/CI needs:

Target	Effect
make / make all	Builds bin/server and bin/client.
make test	Builds everything (including the gtest binaries), then runs unit tests, then the integration test. Non-zero exit on any failure.
make clean	Removes build/ (object/test artifacts).
make fclean	clean, plus removes bin/ (the final binaries).
make re	fclean then all.

Key implementation details:

- server.cpp and client.cpp are each compiled as a single translation unit (everything else is header-only), so the “build” step is literally one g++ invocation per binary; -MMD -MP still generates .d dependency files so editing a shared header (e.g. src/common/trie.hpp) correctly triggers a rebuild of both.
- The gtest binaries link against the distro-provided static libraries (-lgtest -lpthread); no CMake or gtest_discover_tests machinery is needed since there are only two small, self-contained test binaries.
- test-integration depends on all (so make test-integration alone still builds fresh binaries first) and simply shells out to tests/integration_test.sh, passing BIN_DIR so the script knows where to find the binaries it needs to exercise.

4 4. Shared utilities — src/common/

4.1 4.1 protocol.hpp — constants and wire-format helpers

This header intentionally contains **no system calls** — it is pure data and pure functions, which makes it trivial to reason about and (if needed) unit test in isolation from any FIFO/threading concerns.

It defines:

- MAX_KEY_BODY_BYTES = 1024 — the combined key+value size cap.
- GRACEFUL_SHUTDOWN_TIMEOUT_SEC = 5 — the shutdown watchdog budget.
- The ExitCode enum (EXIT_OK=0, EXIT_GENERAL_ERROR=1, EXIT_TOPIC_ERROR=2, EXIT_IPC_ERROR=3), used by both the server (to decide what ERR <code> ... to send back) and the client (to decide its own exit() code when it can’t even reach the server).
- SENTINEL_OFFSET / SENTINEL_TEXT_LINE — the special “the server is shutting down, stop blocking on read()” markers described in the Architecture section of the README.

- `split()` — a tiny space-delimited tokenizer used to parse control lines. It is safe specifically *because* every identifier the protocol ever transmits is restricted to `^[a-zA-Z0-9_.-]{1,32}$` (no spaces), which `is_valid_identifier()` also enforces.
- `write_u32le()` / `read_u32le()` — little-endian 32-bit integer framing used by `--raw` mode.

4.2 4.2 ipc_util.hpp — FIFO plumbing

Thin wrappers around `mkfifo()`, `open()`, `read()`, `write()`:

- `fifo_create(path)` — `unlink()`s any stale node first (so re-running the server or resubscribing a client never fails with `EEXIST` in a confusing way), then `mkfifo(path, 0600)`.
- `write_all()` / `read_all()` — retry on `EINTR` and on partial reads/writes, so callers never have to think about short reads/writes. (Note: the *client*'s subscribe loop deliberately does **not** use these — see §6.2 for why.)
- `read_line()` — reads up to and including a `\n`, used for the line-based control protocol and for text-mode messages.

5 5. Data structures — src/common/

5.1 5.1 hashmap.hpp — the client-metadata hashmap

A textbook separate-chaining hashmap, built to the subject's explicit constraint that this particular piece of state may not use `std::unordered_map`/`std::map`.

```
template <typename Value>
class HashMap {
    // ...
    bool set(const std::string &key, const Value &value);
    bool get(const std::string &key, Value &out) const;
    bool remove(const std::string &key);
    template <typename Fn> void for_each(Fn fn) const;
};
```

- **Hashing:** djb2 (`hash = hash*33 + c`) over the raw bytes of the key.
- **Buckets:** `std::vector<std::vector<Entry>>` — an *array of vectors*, where “array” is the actual hash table and each inner vector is one bucket's chain. A collision (two keys hashing to the same bucket index) is resolved by simply appending to that bucket's vector; lookups scan the (short, on average) chain for a matching key.
- **Growth:** `maybe_rehash()` doubles the bucket array and re-inserts every entry once the load factor (`size / bucket_count`) exceeds 0.75. This keeps `get/set/remove` close to $O(1)$ on average even as the number of connected clients grows over the server's lifetime.
- **Thread safety:** a single `std::mutex` wraps every public method. This table is written from two different threads in practice — the control thread (on `SUBSCRIBE`, to record/resume a client's metadata) and every topic's worker thread (every time it advances a consumer's committed offset) — so it has to be safe under concurrent access, not just correct single-threaded.

It is instantiated once, as `HashMap<ClientMeta> g_client_index` in `server.cpp`, where `ClientMeta` holds `{client_id, topic, offset, prefix, ipc}` — exactly the fields the `info` subcommand's JSON output needs.

5.2 5.2 trie.hpp — the prefix-matching structure

Two cooperating pieces:

```

class PrefixTrie {
public:
    void insert(const std::string &prefix, const std::string &consumer_id);
    void remove(const std::string &prefix, const std::string &consumer_id);
    std::vector<std::string> match(const std::string &key) const;
};

inline bool prefix_matches(const std::string &prefix, const std::string &key);

class PrefixIndex { // PrefixTrie + the empty-prefix ("wildcard") fast path
public:
    void add(const std::string &prefix, const std::string &consumer_id);
    void remove(const std::string &prefix, const std::string &consumer_id);
    std::vector<std::string> match(const std::string &key) const;
};

```

PrefixTrie is a classic character-indexed trie: each node has a `std::unordered_map<char, Node*>` of children and a `std::vector<string>` of consumer ids registered *exactly* at that node (i.e. whose prefix ends there). `insert("user.create", "c1")` walks/creates one node per character of "user.create" and appends "c1" to the final node. `match("user.create.extra")` walks the trie one character of the *key* at a time and, at every node it successfully steps into, collects that node's consumer ids — this is exactly the set of registered prefixes that are a prefix of the key, computed in a single $O(\text{len}(\text{key}))$ pass regardless of how many prefixes are registered in total.

prefix_matches(prefix, key) is the free-standing “ground truth” for what counts as a match (exact match counts; too-long a prefix never matches; an empty prefix matches everything) — used both inside PrefixTrie's own logic conceptually and, directly, by the server's catch-up/replay scan (`Topic::run()`, §6.1) so there is exactly one definition of “match” in the code-base.

PrefixIndex wraps a PrefixTrie and adds the flat “wildcard” list required by the subject's “Direct Addition” rule: a consumer that subscribes with an empty prefix goes straight into a `std::vector<std::string>`, bypassing trie traversal entirely (since it always matches, there's nothing to traverse for).

6. Server — src/server/

6.1 6.1 topic.hpp — one worker thread per topic

Topic owns everything about one topic: its append-only log (`std::vector<Message>`), its registered consumers, and its PrefixIndex. It also owns exactly one `std::thread` (`Topic::start()/run()`), satisfying the subject's “at least one dedicated thread per topic” requirement.

```

uint32_t produce(const std::string &key, const std::string &value);
void register_consumer(const std::string &id, const std::string &prefix,
                      uint32_t start_offset, bool raw);
void attach_fd(const std::string &id, int fd);
void unregister_consumer(const std::string &id);

```

The worker loop (`Topic::run()`), simplified:

```

while (true) {
    cv_.wait_for(lock, 150ms, [this] {

```

```

        return shutting_down_ || has_pending_work_locked();
    });
    if (shutting_down_) break;

    for (auto &[id, c] : consumers_) {
        if (!c.connected) continue;
        while (c.offset < log_.size()) {
            const Message &m = log_[c.offset];
            if (prefix_matches(c.prefix, m.key))
                write_all(c.fd, encode(m)); // text or --raw
            c.offset++;
            persist_cb_(c.id, c.offset);      // -> g_client_index
        }
    }
}
// on shutdown: write a sentinel to every connected consumer, then close it

```

This single loop does double duty:

- **Catch-up**, for a consumer that just subscribed (or resubscribed after a disconnect) with `offset < log_.size()` already — it simply replays every log entry it hasn't seen yet.
- **Live delivery**, for a consumer that's already caught up — a new `produce()` call appends to `log_` and notifies the condition variable, waking the loop to deliver just the new entry.

Because both cases funnel through the same `while (c.offset < log_.size())` loop, there is no separate “backlog replay” code path to keep in sync with the “live push” code path — and, importantly, the offset always advances past *every* log entry (matching or not), which is what makes info's reported offset equal “one past the highest topic offset this consumer has seen” rather than “a count of matching messages” (see the worked example in §8).

6.2 6.2 server.cpp — control loop and request handlers

`main()`:

1. Installs `SIGINT/SIGTERM` handlers that just set an atomic flag, and ignores `SIGPIPE` (so a dead consumer's closed FIFO can never kill the server via a signal — a failed `write()` is handled as a normal error return instead, see `Topic::run()`).
2. Computes the IPC identifier `/tmp/operator.server.<pid>`, creates that FIFO, and opens it `O_RDWR | O_NONBLOCK — O_RDWR` so the server is always its own writer (meaning `read()` never returns `EOF` the moment the last real client closes its end), `O_NONBLOCK` so a `poll()`-based loop can periodically re-check the shutdown flag instead of blocking forever in `read()`.
3. Prints the IPC identifier to `stdout` (this is the one piece of output test tooling needs to parse to start talking to the server).
4. Loops: `poll()` with a 150ms timeout, `read()` whatever arrived, accumulate into a buffer, and `dispatch()` each complete `\n`-terminated line as one control request.

`dispatch()` routes a line's first token to one of six handlers:

Command	Handler	Effect
CREATE <topic> <resp>	handle_create	Validates the name, checks for a duplicate, constructs+starts a new Topic.
LIST <resp>	handle_list	Comma-joins the names of every live topic.
INFO <client_id> <resp>	handle_info	Looks the id up in g_client_index, formats it as JSON.

Command	Handler	Effect
SUBSCRIBE <topic> <id> <prefix -> <offset -1> <raw> <resp>	handle_subscribe	Validates, computes the starting offset (explicit, resumed, or 0), creates the consumer's dedicated FIFO, registers it with the Topic, replies, then spawns a background thread to complete the FIFO open-for-write rendezvous.

Command	Handler	Effect
PRODUCE <topic> <raw> <fifo> <resp>	handle_produce	Validates the topic exists, replies, then spawns a background thread that reads framed records from the producer's FIFO until EOF and calls <code>Topic::produce()</code> for each one.
DISCONNECT <client_id>	handle_disconnect	Fire-and-forget: removes a consumer from its topic immediately (used when a client is interrupted by Ctrl+C rather than waiting for the server to notice a broken pipe).

Every handler that expects an answer is given the path of a client-created response FIFO as its last argument; `reply_ok()`/`reply_err()` simply `open()` that path `O_WRONLY`, write one line, and `close()` it — see §7's "request/response over a one-way pipe" walkthrough.

7 7. Client — `src/client/client.cpp`

All five subcommands share two small building blocks:

- **`request_reply(ipc_id, cmd, &reply)`**: creates a private response FIFO (`/tmp/.operator.resp.<pid>.<cmd>`), sends "`<cmd> <resp_path>`" to the server's main FIFO, blocks on `open(resp_path, O_RDONLY)` + `read_line()`, then deletes the response FIFO. If the server's main FIFO can't even be opened (no such server), the client dies immediately with exit code 1 ("invalid ipc identifier"); if it sends fine but never gets an answer, that's treated as exit code 3 ("server disconnected").
- **`expect_ok_or_die(reply)`**: parses "`OK ...`" vs "`ERR <code> <message>`" and either returns the payload or prints the message to `stderr` and calls `exit(code)` directly — this is what makes every subcommand's exit-code handling a one-liner.

7.1 7.1 create / list / info

Straightforward `request_reply` + `expect_ok_or_die` + print to `stdout`. `info`'s payload is already a ready-to-print JSON string built server-side.

7.2 7.2 produce [--raw]

1. Creates its own private producer FIFO, sends the `PRODUCE` handshake (including whether it's `--raw`), and waits for `OK/ERR` exactly like the commands above — this is where a non-existent topic is caught and turned into exit code 2.
2. Opens the producer FIFO `O_WRONLY` (rendezvous with the server's background reader thread, §6.2).
3. Reads from `stdin` until EOF:
 - **text mode**: one `key:value` line at a time, validated against the 1024-byte combined limit, forwarded verbatim (plus its `\n`).
 - **--raw mode**: `fread()`s a 4-byte little-endian key length, the key bytes, a 4-byte value length, and the value bytes; a short read at any point *except* exactly at a record boundary is treated as a truncated record (exit code 1); otherwise the exact same length-prefixed bytes are forwarded to the producer FIFO.
4. Closes the FIFO on clean EOF and exits 0.

7.3 7.3 subscribe <topic> <name> [--prefix P] [--offset N] [--raw]

1. Sends the `SUBSCRIBE` handshake and waits for `OK/ERR` — a duplicate consumer name currently active on that topic is what turns into exit code 2 here.
2. Prints subscribed to `<topic>` (this happens *before* switching into binary output for `--raw` mode too, matching the subject's example transcript, which shows this same confirmation line for every consumer regardless of mode).
3. Installs `SIGINT/SIGTERM` handlers **without** `SA_RESTART`, then opens its dedicated message FIFO `O_RDONLY` (this is the rendezvous with the server's background "open for write" thread from `handle_subscribe`, §6.2).
4. Loops reading messages (text lines, or raw length-prefixed records) and printing them to `stdout`, until either:

- the server sends the shutdown sentinel (`\x04` line / offset == `0xFFFFFFFF` record) — the loop breaks and the process exits 0 without contacting the server again (it already knows the server is going away);
- Ctrl+C/SIGTERM arrives — see the next paragraph;
- the FIFO otherwise closes/errors — treated the same as a graceful stop.

Why subscribe has its own `read_all_interruptible/ read_line_interruptible` instead of reusing `ipc_util.hpp`'s `read_all/read_line`: the shared helpers deliberately retry on EINTR, because everywhere *else* in the client, a spurious interrupt mid-read should just be retried transparently. But subscribe's main loop specifically relies on Ctrl+C delivering EINTR to a blocked `read()` so it can break out of the loop and check the “was I signaled?” flag — a transparently-retrying read would swallow that interrupt and go straight back to sleep in the kernel, and Ctrl+C would appear to do nothing until the next message arrived (if ever). The interruptible variants return false immediately on EINTR instead of looping, and the caller then checks `g_signaled` to tell “the server hung up” apart from “I was interrupted.” (This was an actual bug found and fixed during development — see the AI usage note in the README.)

When the loop exits because of a signal (not a server-initiated sentinel), the client sends a best-effort DISCONNECT `<client_id>` to the main FIFO before exiting, so the server frees that consumer's slot (and the prefix-trie entry) immediately instead of only noticing on the next failed write().

8. A full request walked end to end

This is the exact transcript from the subject's worked example (VIII.3), reproduced by this implementation:

```
$ ./bin/server
/tmp/operator.server.1337
```

```
$ ./bin/client /tmp/operator.server.1337 create user_events
topic created
```

```
$ ./bin/client /tmp/operator.server.1337 subscribe user_events client0 \
--prefix user.create &
subscribed to user_events
```

```
$ ./bin/client /tmp/operator.server.1337 produce user_events <<'EOF'
user.create:{"user":"reach","age":25}
user.update:{"user":"norminet","age":42}
user.delete:{"user":"froz","age":24}
user.create:{"user":"moulinette","age":1337}
EOF
```

```
# client0 prints:
user.create:{"user":"reach","age":25}
user.create:{"user":"moulinette","age":1337}
```

```
$ ./bin/client /tmp/operator.server.1337 info client0
{"client":"client0","topic":"user_events","offset":4,"prefix":"user.create","ipc":"/tmp/operato
```

Tracing what actually happens:

1. create opens the main FIFO, writes CREATE `user_events <resp>`, `handle_create` builds a new Topic(`"user_events"`, ...) and starts its worker thread, replies OK.

2. `subscribe` writes `SUBSCRIBE user_events client0 user.create -1 0 <resp>`. `handle_subscribe` sees no prior `client0` metadata, so `start_offset = 0`; it registers `client0` in the topic's `PrefixIndex` under `"user.create"`, replies OK, then a background thread blocks in `open(msg_fifo, 0_WRONLY)`. `client0`'s own process is simultaneously blocked in `open(msg_fifo, 0_RDONLY)` — the two `open()` calls rendezvous, the background thread hands the `fd` to `Topic::attach_fd`, and the consumer becomes connected.
3. `produce` streams four `key:value` lines through its own private producer FIFO; the server's reader thread calls `Topic::produce()` four times, assigning offsets 0, 1, 2, 3 and notifying the topic's worker thread each time.
4. The worker thread wakes, sees `client0.offset (0) < log_.size()`, and walks offsets 0→3: offset 0 (`user.create:...reach...`) matches the `"user.create"` prefix and is written to `client0`'s `fd`; offset 1 (`user.update:...`) does not match, but `client0.offset` still advances past it; offset 2 (`user.delete:...`) likewise doesn't match but still advances the offset; offset 3 (`user.create:...moulinette...`) matches and is delivered. After the loop, `client0.offset == 4` and that value has been pushed into `g_client_index` via the `persist` callback after *every* increment (including the two non-matching ones) — which is exactly why `info` reports `"offset":4` and not `"offset":2` (a count of matching messages) or `"offset":3` (the highest matching offset).
5. `info` looks `client0` up in `g_client_index` and formats the stored `ClientMeta` as the JSON line shown above.

9 9. Testing strategy

9.1 9.1 Unit tests (tests/test_trie.cpp, tests/test_hashmap.cpp)

Both use GoogleTest, linked directly against the distro's `-lgtest` static library (each test file defines its own `main()` via `RUN_ALL_TESTS()`, so no external test runner is required). They test the two hand-rolled data structures in isolation from any FIFO/threading concerns:

- `test_trie.cpp` (16 cases): the `prefix_matches()` ground-truth rule in isolation, then `PrefixTrie` (single/shared/nested prefixes, removal, a no-op removal, an empty trie, and a 500-prefix scale check with fixed-width identifiers chosen specifically so no prefix accidentally contains another — see the comment in that test for the `"topic1"-is-a-prefix-of-"topic10"` pitfall that was caught while writing it), then `PrefixIndex` (the wildcard fast path, removing a wildcard consumer, and a mix of wildcard + specific consumers).
- `test_hashmap.cpp` (8 cases): basic set/get/remove, overwrite-vs-insert semantics, forcing every key into a single bucket to directly exercise the chaining collision path, forcing multiple rehashes and checking every entry survives, `for_each` coverage, and an 8-thread concurrent read/write stress test.

9.2 9.2 Integration test (tests/integration_test.sh)

A black-box bash script that starts a real bin/server and drives it purely through real bin/client invocations — the same interface an automated evaluator or a human would use — and checks:

- topic creation, duplicate-topic rejection (exit 2), listing;
- an invalid IPC identifier (exit 1) and producing to a missing topic (exit 2);
- the exact subject worked example (prefix-filtered subscribe + produce + info, byte-for-byte);
- duplicate-consumer-name rejection while a subscriber is active (exit 2);
- a consumer disconnecting and later resubscribing with the same name, verifying it resumes from its last committed offset rather than replaying from the start or missing messages produced in between;

- `--raw` binary mode round-tripping offset/key/value correctly (via `tests/raw_helpers.py`, which builds/parses the exact little-endian framed byte layout so the shell script itself never has to);
- that a consumer blocked in `subscribe` (no traffic) is woken and exits cleanly when the server receives `SIGTERM`, and that the server itself exits promptly rather than requiring the full 5-second watchdog budget.

Every check increments a pass/fail counter and the script's own exit code is non-zero if any check failed, which is what lets `make test` propagate failure correctly.

10 10. Exit codes reference

Code	Meaning	Example trigger
0	Success	Any command completing normally; a <code>subscribe</code> process exiting after a graceful server shutdown or a local <code>Ctrl+C</code> .
1	General error	Malformed CLI arguments; an invalid topic/client identifier; the IPC identifier does not correspond to a running server; a truncated <code>--raw</code> record at EOF.
2	Topic/client error	Creating a topic that already exists; producing/subscribing to a topic that doesn't exist; subscribing with a name already active on that topic; info on an unknown client id.
3	IPC/communication error	The server accepted a request but never answered (e.g. crashed mid-response).

11 11. Known limitations and deliberate trade-offs

- **Offset commit is implicit, not acknowledged.** A message is considered “committed” the moment the topic's worker thread successfully `write()`s it to the consumer's FIFO, not after an explicit application-level acknowledgement from the client. This keeps the protocol simple (no extra ACK round trip needed per message) at the cost of an at-most-once rather than a fully-acknowledged delivery guarantee: if a consumer crashes after the kernel has buffered a message into its pipe but before the consumer's own code has processed/printed it, that message is not redelivered on resubscribe.

- **Duplicate-consumer detection is per-topic and in-memory only.** A consumer name is only rejected as a duplicate while it has an *active* registration on that specific topic; the same name can be reused on a different topic, or reused on the same topic once the prior session has disconnected.
- **The main control FIFO's atomicity relies on PIPE_BUF.** This is a standard, portable-on-Linux guarantee (4096 bytes), but the protocol is deliberately conservative about it — every field is drawn from the 32-character identifier charset with no spaces, keeping every control line far under that limit even with several fields concatenated.